DESENVOLVIMENTO MOBILE · MÓDULO 08 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

Arquitetura e Gerência de Estado

Arquitetura recomendada (MVVM), views, view models, repositórios, serviços, Result, Command, injeção de dependência, Provider, Riverpod e Bloc

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Aplicar a arquitetura recomendada pela documentação do Flutter: camada de UI e camada de dados.
- Implementar *views*, *view models*, repositórios e serviços.
- Tratar falhas com o tipo `Result` e ações assíncronas com o padrão **Command**.
- Injetar dependências com **Provider**.
- Comparar Provider, Riverpod e Bloc e escolher a abordagem adequada.
- Organizar as pastas de um app que vai crescer.

Roteiro

Neste módulo

- Fundamentos: arquitetura de software
- A arquitetura recomendada
- Camada de dados
- Camada de UI
- Injeção de dependência
- Gerência de estado: alternativas
- Fechamento

Fundamentos: arquitetura de software

O que é arquitetura de software

Arquitetura

As grandes estruturas de um sistema: seus elementos, as relações entre eles e as decisões **difíceis de mudar depois**. Ela existe para proteger atributos de qualidade.

Atributo	Pergunta no LabMonitor
Manutenibilidade	trocar a API exige mexer nas telas?
Testabilidade	dá para testar a regra de reserva sem um aparelho?
Desempenho	a lista de leituras rola sem engasgar?
Funcionamento *offline*	o app mostra dados sem internet?
Segurança	o *token* fica protegido?

Analogia

A planta de uma casa decide por onde passam os canos antes da primeira parede. Pintar um quarto não é arquitetura; mudar a cozinha de lugar é.

Boa arquitetura em um app

Permite mudar a interface sem tocar nos dados, trocar a API sem tocar na interface e testar regras sem abrir o app.

Dica

Coesão, acoplamento, SOLID e padrões de projeto foram vistos no Módulo 03: aqui eles ganham escala.

Padrões de apresentação: de MVC a MVI

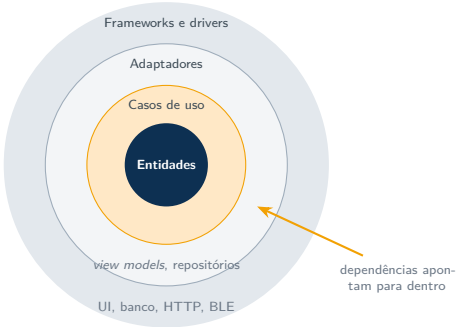


Padrão	Quem atualiza a tela	Ponto forte	Ponto fraco
MVC	o *controller* manipula a *view*	simples	no Android, a `Activity` vira um *controller* gigante
MVP	o *presenter* chama métodos de uma interface da *view*	testável sem a tela	muito código de interface; guarda referência à *view*
MVVM	a *view* observa o estado exposto pelo *view model*	casa com UI declarativa e sobrevive à rotação	vários fluxos de estado podem se desencontrar
MVI	estado único e imutável; cada intenção gera um novo estado	previsível e fácil de depurar	cerimônia demais para telas simples

Na prática atual

Compose e Flutter usam MVVM com um estado único por tela (`UiState`): uma mistura de MVVM com a disciplina do MVI.

Arquitetura limpa e arquitetura hexagonal



Regra da dependência

O código das camadas internas não conhece as externas: as regras de negócio não importam nada de Android, Flutter, HTTP ou SQL.

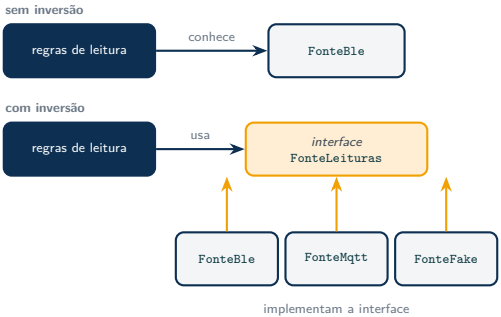
Um pouco de história

Alistair Cockburn propôs a arquitetura hexagonal, de portas e adaptadores, em 2005. Robert C. Martin reuniu ideias parecidas na Arquitetura Limpa, em 2012.

Na medida certa

Em apps pequenos, camadas de UI e de dados bastam. Crie a camada de domínio quando regras se repetirem ou precisarem de testes isolados, como recomendam o Android e o Flutter.

Inversão de dependência na prática



Dart

```dart
abstract interface class FonteLeituras {
  Stream<Leitura> leituras();
}
```

Inversão de dependência

Módulos de alto nível não dependem de detalhes; ambos dependem de abstrações. Quem precisa define a interface; a camada de dados a implementa.

Kotlin

```kotlin
interface FonteLeituras {
  fun leituras(): Flow<Leitura>
}
```

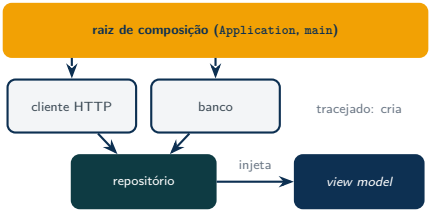
O que se ganha

Trocar BLE por MQTT sem mexer nas regras, testar com uma fonte falsa e fazer a tela antes de o hardware existir.

Injeção de dependência e localizador de serviços

Inversão de controle

A classe não cria o que usa: alguém de fora monta os objetos e os entrega. É o princípio de Hollywood: "não nos chame, nós chamamos você".



Um pouco de história

Martin Fowler batizou o padrão de injeção de dependência em 2004.

Abordagem	Como funciona
Construtor	recebe tudo pronto; dependências explícitas e fáceis de trocar em testes
Contêiner	uma biblioteca monta o grafo: Hilt, Koin; Provider, Riverpod
Localizador	a classe pede a um registro global (`get_it`): prático, mas esconde dependências e só falha em execução

Tempo de vida (escopo)

Único no app: cliente HTTP, banco. **Por tela**: *view model*. **Por uso**: objeto novo a cada pedido. Escopo errado causa vazamento ou estado compartilhado por engano.

Decisões de arquitetura: trocas e registros

Toda decisão é uma troca

Mais camadas trazem isolamento e testabilidade, mas também mais arquivos e indireção. Pese o tamanho do time, a vida esperada do app e o custo de errar.

Princípio	Significado
YAGNI	não construa o que ainda não é necessário
KISS	prefira a solução mais simples que funciona
DRY	cada conhecimento em um só lugar
Regra de três	abstraia na terceira repetição, não na primeira

ADR (*Architecture Decision Record*)

Texto curto, versionado com o código, que registra uma decisão: contexto, opções, decisão e consequências. Responde no futuro: "por que fizemos assim?".

ADR 007 — Estado das telas

Contexto: leituras do sensor em tempo real. **Decisão**: um `UiState` imutável por tela, exposto como fluxo observável. **Consequência**: telas testáveis sem interface.

Um pouco de história

Michael Nygard propôs os ADRs em 2011.

A arquitetura recomendada

Por que arquitetura?



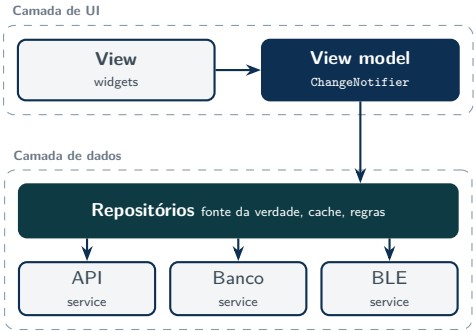
Tudo dentro do widget

- Impossível testar regras sem montar a tela.
- Uma mudança na API quebra várias telas.
- Dados duplicados e inconsistentes entre telas.

Princípios

- Separação de responsabilidades.
- Fonte única da verdade.
- Fluxo unidirecional de dados.
- UI como função do estado.

Camadas: UI e dados



Peça	Responsabilidade
View	desenhar e repassar eventos; sem lógica
View model	transformar dados em estado de tela e expor ações
Repositório	fonte da verdade: cache, erros, retentativas
Serviço	falar com uma fonte externa (HTTP, SQLite, BLE)

Dica

Uma view para cada view model. Repositórios e serviços são compartilhados entre telas.

Organização das pastas

lib/

```
lib/
|-- main.dart
|-- routing/router.dart
|-- domain/
|   `-- models/         bancada.dart, leitura.dart
|-- data/
|   |-- services/       api_service.dart, ble_service.dart
|   `-- repositories/   bancada_repository.dart
|-- ui/
|   |-- core/           tema e widgets comuns
|   |-- painel/
|   |   |-- view_models/painel_view_model.dart
|   |   `-- widgets/painel_screen.dart
|   `-- equipamentos/
`-- utils/              result.dart, command.dart
test/                   espelha lib/
```

Organização mista

A camada de UI é organizada **por funcionalidade** (cada tela tem sua view e seu view model). A camada de dados é organizada **por tipo**, porque repositórios e serviços são usados por várias telas.

Referência oficial

O app de exemplo **Compass** (`flutter/samples`) segue exatamente esta estrutura e é o estudo de caso da documentação.

Camada de dados

Tratando falhas com `Result`

utils/result.dart

```dart
sealed class Result<T> {
  const Result();
}

final class Ok<T> extends Result<T> {
  const Ok(this.valor);
  final T valor;
}

final class Falha<T> extends Result<T> {
  const Falha(this.erro);
  final Exception erro;
}
```

Usando

```dart
final r = await repo.listar();
switch (r) {
  case Ok(:final valor):
    bancadas = valor;
  case Falha(:final erro):
    mensagem = traduzir(erro);
}
```

Por que `Result`?

O tipo de retorno deixa explícito que a operação pode falhar e o `switch` exaustivo obriga quem chama a tratar o erro. Exceções esquecidas viram telas vermelhas.

Serviços e repositórios

data/repositories/bancada_repository.dart

```dart
abstract class BancadaRepository {
  Future<Result<List<Bancada>>> listar();
}

class BancadaRepositoryRemoto implements BancadaRepository {
  BancadaRepositoryRemoto({required this.api});
  final ApiService api;
  List<Bancada>? _cache;

  @override
  Future<Result<List<Bancada>>> listar() async {
    if (_cache != null) return Ok(_cache!);
    try {
      final dtos = await api.getBancadas();   // serviço
      _cache = dtos.map((d) => d.paraModelo()).toList();
      return Ok(_cache!);
    } on Exception catch (e) { return Falha(e); }
  }
}
```

Serviço

Envolve uma fonte externa e devolve dados brutos (DTOs). Não guarda estado nem decide nada.

Repositório

Transforma DTOs em modelos de domínio, mantém cache, trata erros e decide de onde vêm os dados.

Dica

A classe abstrata permite trocar a implementação (remota, local ou um *fake* nos testes) sem mexer na UI.

Camada de UI

O padrão Command

utils/command.dart (simplificado)

```dart
class Command0<T> extends ChangeNotifier {
  Command0(this._acao);
  final Future<Result<T>> Function() _acao;

  bool executando = false;
  Result<T>? resultado;
  bool get falhou => resultado is Falha;

  Future<void> execute() async {
    if (executando) return;          // evita duplo clique
    executando = true;
    resultado = null;
    notifyListeners();
    try {
      resultado = await _acao();
    } finally {
      executando = false;
      notifyListeners();
    }
  }
}
```

Command

Envolve uma ação assíncrona do view model e expõe seu andamento: **executando**, **concluída** ou **falhou**. A view desenha cada estado sem `bool`s espalhados.

Dica

É o padrão usado no estudo de caso oficial do Flutter. Existe também a variante `Command1<T, A>` para ações com argumento.

O view model

ui/painel/view_models/painel_view_model.dart

```dart
class PainelViewModel extends ChangeNotifier {
  PainelViewModel({required this.repo}) {
    carregar = Command0(_carregar)..execute();
  }
  final BancadaRepository repo;
  late final Command0<List<Bancada>> carregar;

  List<Bancada> _bancadas = [];  String _filtro = '';
  List<Bancada> get visiveis => _bancadas
      .where((b) => b.nome.contains(_filtro)).toList();
  void filtrar(String f) { _filtro = f; notifyListeners(); }

  Future<Result<List<Bancada>>> _carregar() async {
    final r = await repo.listar();
    if (r case Ok(:final valor)) _bancadas = valor;
    notifyListeners();
    return r;
  }
}
```

- Recebe os repositórios no construtor (injeção).
- Guarda o estado da tela e o expõe só para leitura.
- Ações são métodos ou *commands*.
- Não importa `flutter/material.dart`: é Dart puro, fácil de testar.

Atenção

O view model nunca guarda `BuildContext` nem widgets.

A view

ui/painel/widgets/painel_screen.dart

```dart
class PainelScreen extends StatelessWidget {
  const PainelScreen({super.key, required this.vm});
  final PainelViewModel vm;

  @override
  Widget build(BuildContext context) => ListenableBuilder(
    listenable: Listenable.merge([vm, vm.carregar]),
    builder: (context, _) {
      if (vm.carregar.executando) {
        return const Carregador();   // spinner centralizado
      }
      if (vm.carregar.falhou) {
        return ErroComRetentativa(
            onTentar: vm.carregar.execute);
      }
      return ListaBancadas(
          bancadas: vm.visiveis, onFiltro: vm.filtrar);
    },
  );
}
```

Responsabilidade da view

Ler o estado do view model e desenhar; repassar eventos chamando métodos e *commands*. Nenhuma regra de negócio.

Teste fácil

A tela recebe o view model no construtor: num teste, passe um view model com um repositório *fake* (Módulo 14).

Injeção de dependência

Injetando com Provider

main.dart

```dart
void main() {
  runApp(MultiProvider(
    providers: [
      Provider(create: (_) => ApiService()),
      Provider(create: (_) => BancoService()),
      Provider<BancadaRepository>(create: (ctx) =>
          BancadaRepositoryRemoto(api: ctx.read())),
      ChangeNotifierProvider(create: (_) => AuthRepository()),
    ],
    child: const LabMonitorApp(),
  ));
}

// na rota: cria o view model com as dependências
GoRoute(path: '/painel', builder: (ctx, _) =>
    PainelScreen(vm: PainelViewModel(repo: ctx.read())));
```

Provider

Pacote recomendado pela documentação do Flutter para disponibilizar objetos na árvore de widgets. `context.read<T>()` obtém o objeto; `context.watch<T>()` também reconstrói quando ele notifica.

Dica

Alternativas de DI: construtores (manual), `get_it` (*service locator*) ou o próprio Riverpod.

Gerência de estado: alternativas

Riverpod

Painel com Riverpod

```
final repoProvider = Provider<BancadaRepository>(
    (ref) => BancadaRepositoryRemoto(api:
      ref.watch(apiProvider)));

final bancadasProvider = FutureProvider<List<Bancada>>(
    (ref) => ref.watch(repoProvider).listarOuFalhar());

class PainelScreen extends ConsumerWidget {
  const PainelScreen({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final estado = ref.watch(bancadasProvider);
    return estado.when(
      loading: () => const CircularProgressIndicator(),
      error: (e, _) => Text('Erro: $e'),
      data: (lista) => ListaBancadas(bancadas: lista),
    );
  }
}
// runApp(const ProviderScope(child: LabMonitorApp()));
```

Riverpod

Providers globais e tipados, independentes da árvore de widgets. Cache, carregamento e erro (`AsyncValue`) já embutidos; dependências entre providers com `ref.watch`.

Dica

`NotifierProvider` faz o papel do view model com estado mutável; geração de código com `@riverpod` reduz repetição.

Bloc / Cubit

Painel com Cubit

```
sealed class PainelEstado {}
class Carregando extends PainelEstado {}
class Pronto extends PainelEstado {
  Pronto(this.bancadas); final List<Bancada> bancadas; }
class ErroPainel extends PainelEstado {
  ErroPainel(this.msg); final String msg; }

class PainelCubit extends Cubit<PainelEstado> {
  PainelCubit(this._repo) : super(Carregando());
  final BancadaRepository _repo;

  Future<void> carregar() async {
    emit(Carregando());
    switch (await _repo.listar()) {
      case Ok(:final valor): emit(Pronto(valor));
      case Falha(:final erro): emit(ErroPainel('$erro'));
    }
  }
}
```

Na view

```
BlocProvider(
  create: (ctx) =>
      PainelCubit(ctx.read())..carregar(),
  child: BlocBuilder<PainelCubit,
      PainelEstado>(
    builder: (ctx, e) => switch (e) {
      Carregando() => const Carregador(),
      Pronto(:final bancadas) =>
          ListaBancadas(bancadas: bancadas),
      ErroPainel(:final msg) => Text(msg),
    }),
);
```

Bloc

Estados imutáveis emitidos em sequência; `Bloc` completo recebe **eventos**, `Cubit` recebe chamadas de método. Muito usado em equipes grandes.

Qual escolher?

	ChangeNotifier + Provider	**Riverpod**	**Bloc / Cubit**
Origem	SDK do Flutter + pacote recomendado	comunidade (autor do Provider)	comunidade (Very Good Ventures)
Curva	baixa	média	média
Boilerplate	baixo	baixo (com geração)	médio
Pontos fortes	simples, oficial, MVVM direto	cache e assíncrono embutidos, testável	previsível, rastreável, padrão rígido
Indicado para	começar e apps pequenos e médios	apps médios e grandes	equipes grandes, regras complexas

O que importa é a **separação em camadas**: com ela, trocar a biblioteca de estado afeta só a camada de UI. Neste curso usamos **ChangeNotifier + Provider**, como na documentação oficial.

Fechamento

Resumo

Resumo

- Duas camadas: UI (views + view models) e dados (repositórios + serviços).
- Views desenham; view models guardam estado e ações; repositórios são a fonte da verdade.
- `Result` torna as falhas explícitas; **Command** expõe o andamento das ações.
- Dependências entram pelo construtor; Provider as disponibiliza na árvore.
- Riverpod e Bloc são alternativas maduras; a arquitetura em camadas vale para todas.

Glossário (1/2)

Termo	Significado
Arquitetura	estruturas do sistema e decisões difíceis de mudar depois
Camada	grupo de código com uma responsabilidade: UI ou dados (e domínio)
MVVM	a *view* observa o estado exposto pelo *view model*
MVI	estado único e imutável; cada intenção do usuário gera um novo estado
View model	transforma dados em estado de tela e expõe ações
Repositório	fonte única da verdade de um tipo de dado
Serviço	classe que fala com uma fonte externa: HTTP, SQLite, BLE
`Result`	tipo que representa sucesso (`Ok`) ou falha (`Falha`) sem exceções
Command	envolve uma ação e expõe seu andamento: executando, concluída, falhou

Glossário (2/2)

Termo	Significado
Regra da dependência	camadas internas não conhecem as externas
Inversão de dependência	depender de abstrações, não de implementações
Injeção de dependência	receber as dependências prontas em vez de criá-las
Raiz de composição	único lugar onde os objetos concretos são criados e ligados
Localizador de serviços	registro global ao qual a classe pede dependências (`get_it`)
Provider e Riverpod	injeção e gerência de estado pela árvore ou por provedores
Bloc e Cubit	estado emitido como fluxo, a partir de eventos ou de métodos
ADR	registro curto de uma decisão de arquitetura e de seus motivos

Questões de revisão

1. Que problemas aparecem quando um widget faz HTTP, SQL e regras de negócio? Cite três.
2. Compare MVP e MVVM: quem atualiza a tela em cada um?
3. Qual a diferença de responsabilidade entre repositório e serviço?
4. Enuncie a regra da dependência e dê um exemplo de violação.
5. Como a interface `FonteLeituras` permite trocar BLE por MQTT sem mexer no *view model*?
6. Qual a diferença entre injeção de dependência e localizador de serviços? Onde fica o `get_it`?
7. Por que devolver `Result` em vez de lançar exceções nos repositórios?
8. Que critérios usar para escolher entre Provider, Riverpod e Bloc?

Exercícios

Exercício 1 — Refatoração

Pegue uma tela com lógica dentro do widget e separe-a em view, view model, repositório e serviço.

Exercício 2 — Command

Implemente `Command1<T, A>` e use-o na ação "reservar bancada", mostrando um indicador no botão enquanto executa e uma *snackbar* em caso de falha.

Exercício 3 — Comparação

Implemente o mesmo painel com Riverpod e com Cubit. Compare quantidade de código, testabilidade e clareza.

LabMonitor — Etapa 8

Reorganize o app nas pastas recomendadas: `PainelViewModel`, `EquipamentosViewModel`, `BancadaRepository` com implementação *fake* e injeção com Provider.

Para aprofundar

- Guia de arquitetura: `https://docs.flutter.dev/app-architecture/guide`
- Estudo de caso (app Compass): `https://docs.flutter.dev/app-architecture/case-study`
- Recomendações de arquitetura: `https://docs.flutter.dev/app-architecture/recommendations`
- Provider: `https://pub.dev/packages/provider` Riverpod: `https://riverpod.dev`
- Bloc: `https://bloclibrary.dev`

Respostas das questões de revisão

1. As regras não podem ser testadas sem montar a tela; uma mudança na API quebra várias telas; os dados ficam duplicados e inconsistentes; o arquivo fica enorme.
2. No MVP, o *presenter* chama métodos de uma interface da *view*. No MVVM, o *view model* expõe estado e a *view* o observa, sem que ele a conheça.
3. O serviço só conversa com uma fonte externa (HTTP, SQLite, BLE). O repositório combina serviços, decide cache e retentativas e é a fonte da verdade para a UI.
4. As camadas internas não conhecem as externas. Violação: um repositório que importa `package:flutter/material.dart` ou recebe um `BuildContext`.
5. O *view model* depende só da interface; a raiz de composição injeta `FonteBle` ou `FonteMqtt`, e os testes usam uma `FonteFake`.
6. Na DI, a classe recebe as dependências de fora, explícitas no construtor. No localizador, ela mesma as pede a um registro global, como o `get_it`, o que as esconde e só falha em execução.
7. O tipo de retorno deixa claro que a operação pode falhar e o `switch` exaustivo obriga a tratar o erro; exceções esquecidas viram telas vermelhas.
8. Tamanho do time e do app, familiaridade, necessidade de rastrear eventos (Bloc), verificação em compilação e testes (Riverpod) e simplicidade (Provider).

Fim do Módulo 08

Próximo: Módulo 09 (comum) — Fundamentos de banco de dados: modelagem, SQL, normalização, transações e índices.